

1. Define the scope of your societal problem (what should the database be able to answer)?

The scope of our societal problem is defined by the people who have suffered the most during catastrophic events, such as earthquakes. And our goal is to provide a database system with the help of which we can supply affected populations with everything needed to enable a better recovery, and help prevent innocent people from dying.

2. List how the data interacts in short sentences

- A person can be transferred to one or more hospitals, and hospitals can receive many people
- A person may need multiple types of supplies, and one type of supply can be needed by many people
- A hospital can treat many people, while one person can receive treatment from more than one hospital
- A person can have multiple safety-status updates over time, but each safety status update belongs to one person

3. Identify the entities ("nouns", or core objects of your database)

Person, Safety-Update, Hospital, Supplies, Patient-Record, Supply-Order, Patient_Allergy, Patient_Medication

4. Identify the relationships ("verbs", connections between entities)

Person → has → Patient_Record (one-to-many)

Hospital → holds → Patient-Record (one-to-many)

Person → has → Safety-Update(one-to-many)

Patient_Record → has → Patient_Allergy (one-to-many)

Patient_Record → has → Patient_Medication (one-to-many)

Person → submits → Supply_Order (one-to-many)

Supply → assigned to → Supply_Order (one-to-many)

5. Decide which attributes to include per entity ("columns" per Table)

For people (people from different locations - how far away they are from the incident), we need to know their personal information (phone number, emergency contact, address, has children or not, marital status, person_id etc)

For Hospital, we need to keep track of people's medical conditions (allergies, needed drugs), the availability of places in a hospital ..., (hospital_id)

For safety, we need to know whether a person is in a safe space or there is a need to relocate them to shelter. (electricity, gas, access to clean water)(update_id)

For supplies, we keep track of the number of supplies left, what supplies we have and their id

For Patient-Record we need record_id, person_id, hospital_id, admission_date, discharge_date, and medical condition. We also need the date and time of admission and discharge, medical condition, allergies and prescribed medication.

For Patient_Allergy, we need allergy_id, record_id, and allergy_name.

For Patient_Medication we need medication_id, record_id and medication_name

For Supply-Order we need order_id, person_id, Supply_id. We also need the quantity of the order, the time of the order and the status of the order

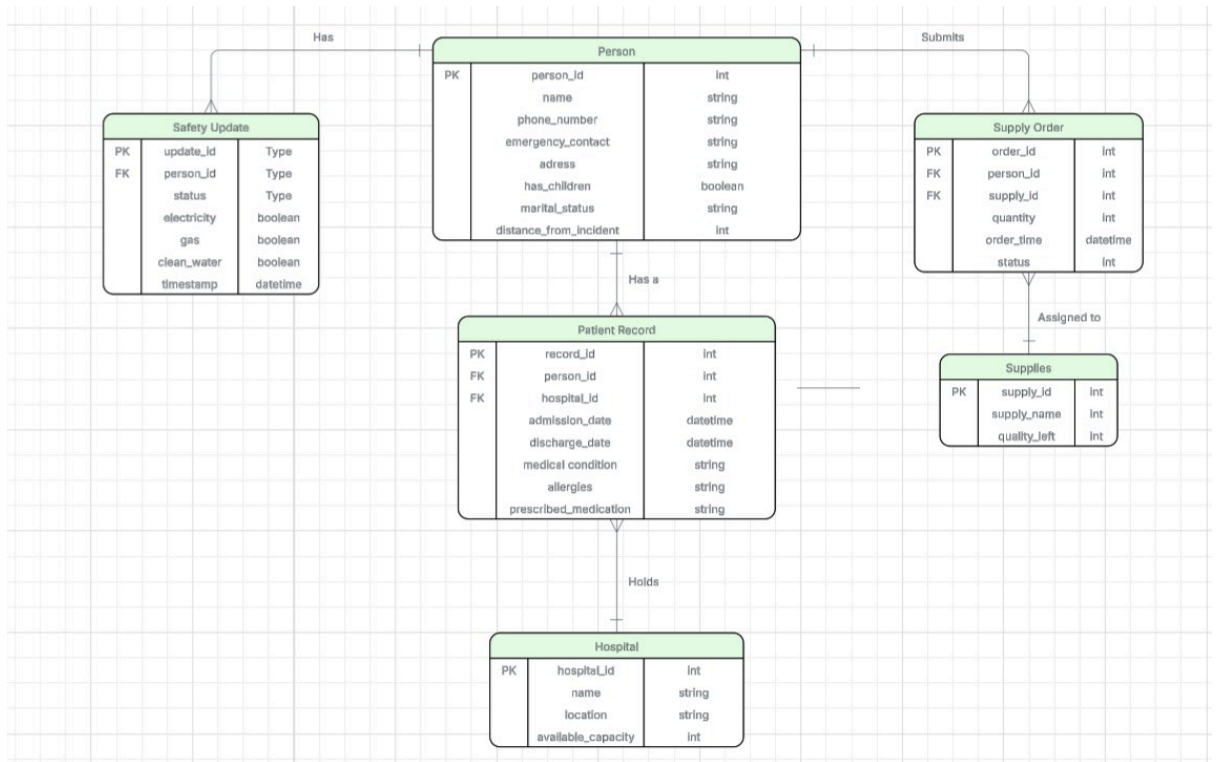
6. Define your PKs and FKs

Each entity requires a unique Primary Key

Foreign Keys are required to link child tables back to parent tables

- Person: PK = person_id
- Hospital: PK = hospital_id
- Supplies: PK = supply_id
- Safety-Update: PK = update_id FK = person_id
- Patient-Record: PK = record_id FK = person_id FK= hospital_id
- Supply-Order: PK = order_id FK = person_id FK = supply_id
- Patient_Allergy: PK = allergy_id, FK = record_id
- Patient_Medication: PK = medication_id, FK = record_id

7. Draw a draft ERD in Crow's foot notation (using any tool you prefer)



Normalize the ERD:

1NF: Atomic Values

Violation Identified: In our initial draft, the Patient_Record entity multi-valued attributes(allergies and prescribed_medication).

Applied Fix: We separated these multi-valued attributes into two new child entities and link them back to Patient_Record using the foreign key record_id.

[Mock data]

1. Patient_Record(Unnormalized)

record_id	person_id	hospital_id	medical_condition	allergies	prescribed_medication	admission_date	discharge_date
101	1	6	Fracture	Penicillin, Dust	Ibuprofen, Paracetamol	2026-09-08 10:00	2026-09-12 14:00
102	2	6	Asthma	Pollen	Albuterol	2026-09-09 11:30	2026-09-10 09:00

2. Patient_Record(After normalization)

record_id	person_id	hospital_id	medical_condition	admission_date	discharge_date
101	1	6	Fracture	2026-09-08 10:00	2026-09-12 14:00
102	2	6	Asthma	2026-09-09 11:30	2026-09-10 09:00

Table: Patient_Allergy(New Entity)

allergy_id	record_id	allergy_name
1	101	Penicillin
2	101	Dust
3	102	Pollen

Table: Patient_Medication(New Entity)

medication_id	record_id	medication_name
1	101	Ibuprofen
2	101	Paracetamol
3	102	Albuterol

2NF: No Partial Dependencies

Our database already satisfies 1NF, and to check whether it satisfies 2NF, we examined whether any non-key attributes depend on only part of a primary key. In our database design, each table uses a single-column primary key, for example person_id, record_id, order_id, hospital_id, supply_id, medication_id and allergy_id. And because none of the

tables use a composite PK, partial dependencies cannot occur. Therefore, our database design satisfies 2NF.

3NF: No Transitive Dependencies

Our database does satisfy 2NF and to satisfy 3NF we checked if any non-key columns depend on other non-key columns, instead of directly depending on PK. But no violations were found, so we can state that our database does satisfy 3NF. For example **Hospital** entity, primary key would be **hospital_id**, and name, location, **available_capacity**, are connected to PK. Same applies to **Person**, **Safety_Update**, **Patient_Record**, **Supply_Order**, etc.

Assignment Submission:

Explain in min. 1 A4 how and why you performed database normalization.

Show the "before" tables with the violation clearly indicated (e.g. with a few rows of mock data), and the after Table(s) showing the applied fix.

